PROGRESS UPDATE · JUNE-JULY 2026

Test Fixture GUI

Bench control software for the sensor-team test fixture

EM · Calibration · Manual · Shear · Fatigue

Jacqueline Chuang

OVERVIEW

What it is

A single browser-based app an operator uses at the bench to run the sensor team's mechanical tests - driving a precision actuator into a sensor while reading a load cell, with every test, safety check, and result in one place.

GUI

Operator's laptop
or the bench PC

→

Hardware

Zaber actuator +
FUTEK load cell

→

Sensor

The sensor
under test

Closed-loop control: the GUI moves the actuator based on the live load-cell reading.

OVERVIEW

Project goals

A platform for configuring, executing, monitoring, and analyzing sensor characterization tests, while keeping ease of use, operator safety, data integrity, and workflow efficiency front and center.

1. **Intuitive user experience**
   Easy to learn and operate with minimal training.

2. **Safe hardware operation**
   Reduces the risk of accidental misuse or unsafe actuator movement.

3. **Data integrity & traceability**
   Every test organized, reproducible, and tied to a sensor and configuration.

4. **Efficiency**
   Minimizes repetitive actions and streamlines characterization.

5. **Clear test monitoring**
   Test progress and system status are understandable in real time.

6. **Multiple test modes**
   Supports EM, Shear, Manual, and Cyclical testing.

7. **Extensibility**
   New tests and analysis tools can be added without a major redesign.

8. **Maintainability**
   Organized so future developers can understand, modify, and extend it.

STATUS

Where the project is

5

test workflows

EM, Calibration, Manual, Shear, and Fatigue

6

safety systems

Force, spike, travel, disconnect, snap-to, single-owner reader

100+

user stories

Across 11 features; test cases done for 7 of 11

- MVP built and running end to end - every test type completes.
- Packaged as a double-clickable desktop app on Mac and Windows.

PART 1

What I built

CAPABILITIES

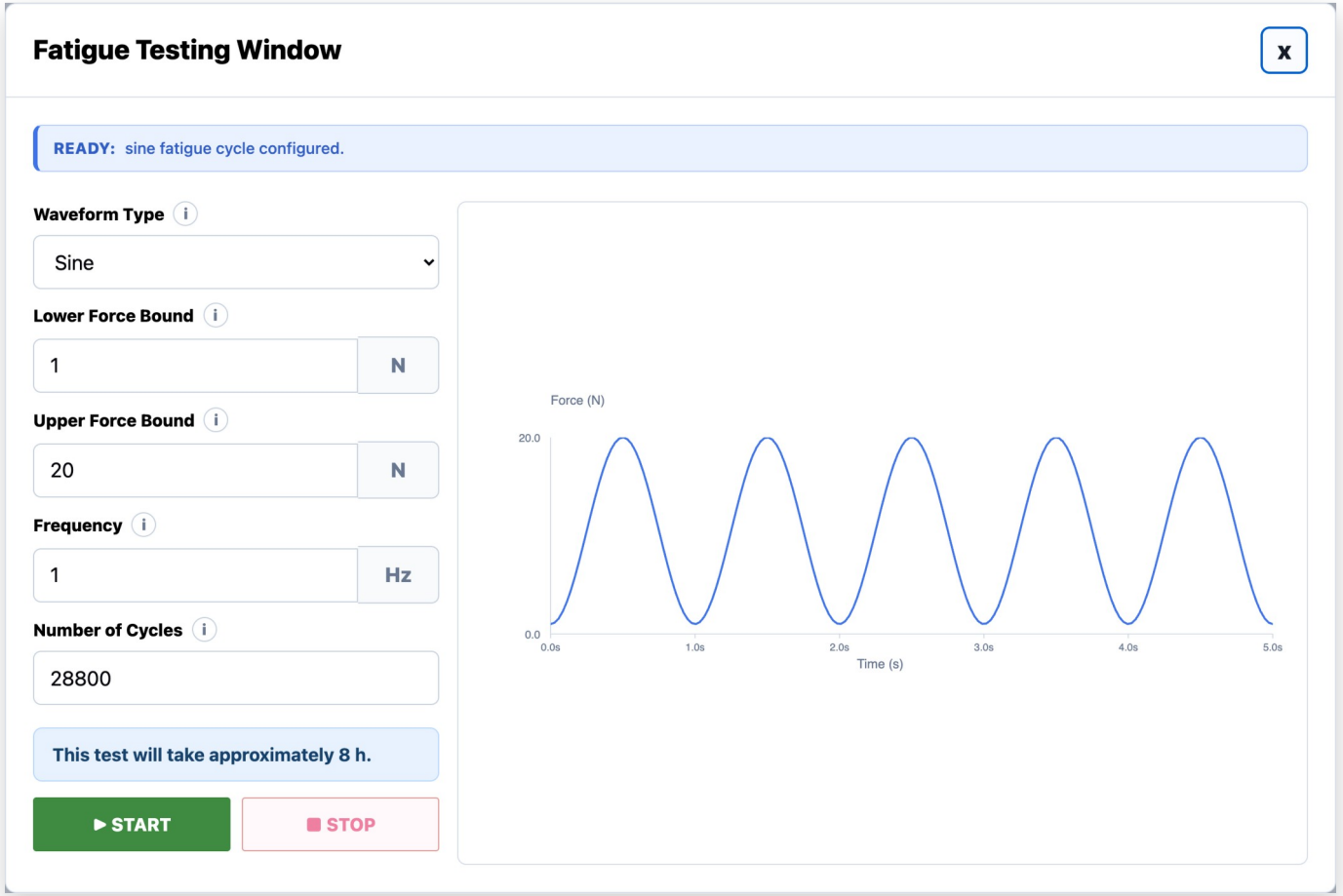
Five test workflows

1. **EM** — Press the sensor and read its response against pressure, automated across repeated runs.
2. **Calibration / Fuji** — Fuji-film calibration with an operator-set extrusion distance and a fine manual jog.
3. **Manual** — Free actuator control by distance or force, with live force recording for exploratory tests.
4. **Shear** — Load-cell-only capture - the operator applies force by hand while the GUI records it.
5. **Fatigue** — Cyclical loading on a choice of waveforms - sine, square, triangle, sawtooth, or a blood-pressure pulse - for thousands of cycles, with a live preview.

DEMO

Fatigue testing window

- **Five waveform types**
  Sine, square, triangle, sawtooth, and a blood-pressure pulse, each redrawn live as the bounds and frequency change.
- **Whole-number force bounds**
  Lower and upper force, snapped into the safe range on entry.
- **Cycle count + duration**
  Shows the estimated test length - here, ~8 hours for 28,800 cycles.
- **Force-spike protection**
  Stops safely if the load ever jumps beyond the configured swing.



CONCEPT

Fatigue waveform builder

- **Presets, no math**

  Sine, square, triangle, sawtooth, or a blood-pressure pulse: pick one and go.

- **Tunable by slider**

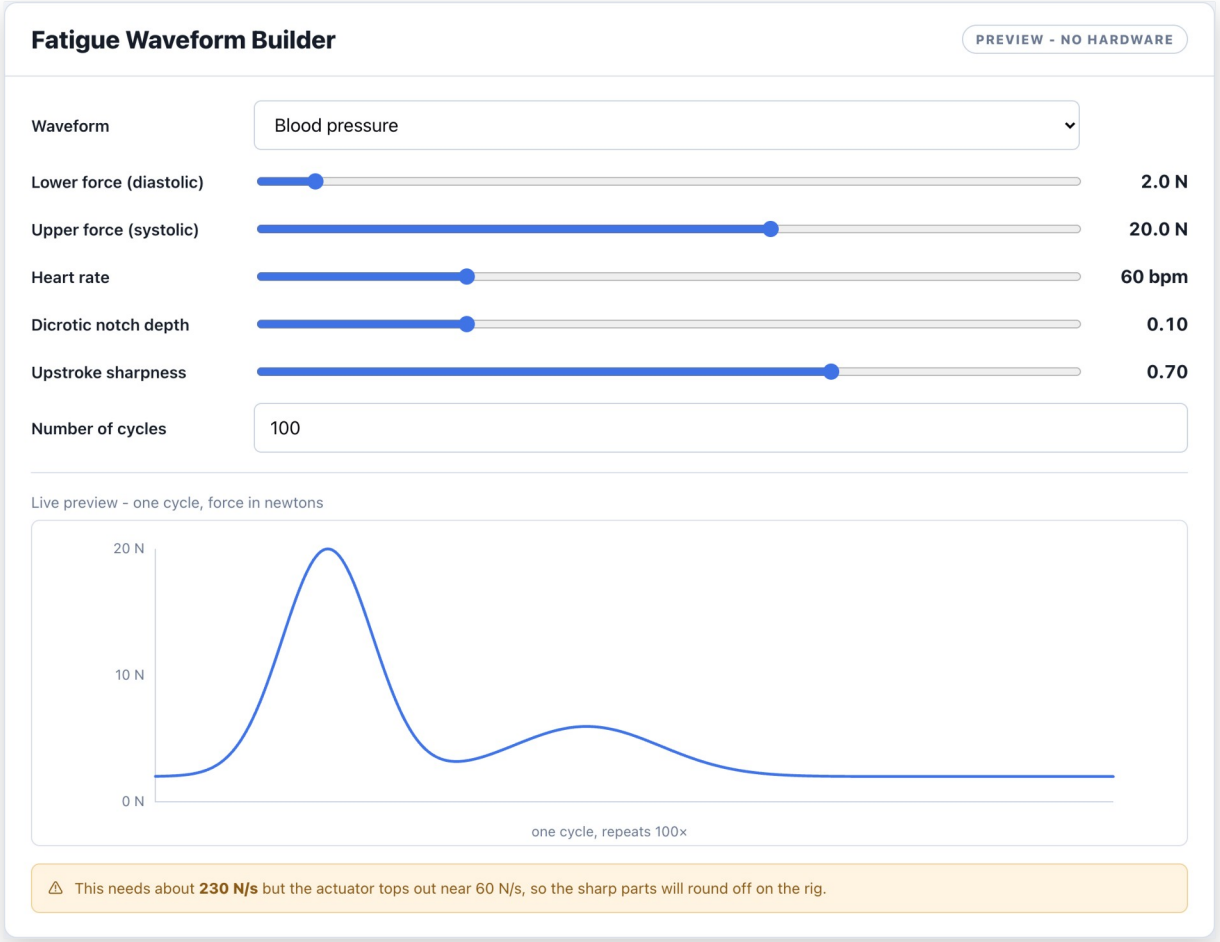
  Blood pressure adds heart rate, dicrotic notch depth, and upstroke sharpness; the sliders write the equation.

- **Too-fast-to-track warning**

  Flags any shape that changes force faster than the actuator can physically follow, before it runs.

- **Custom equation escape hatch**

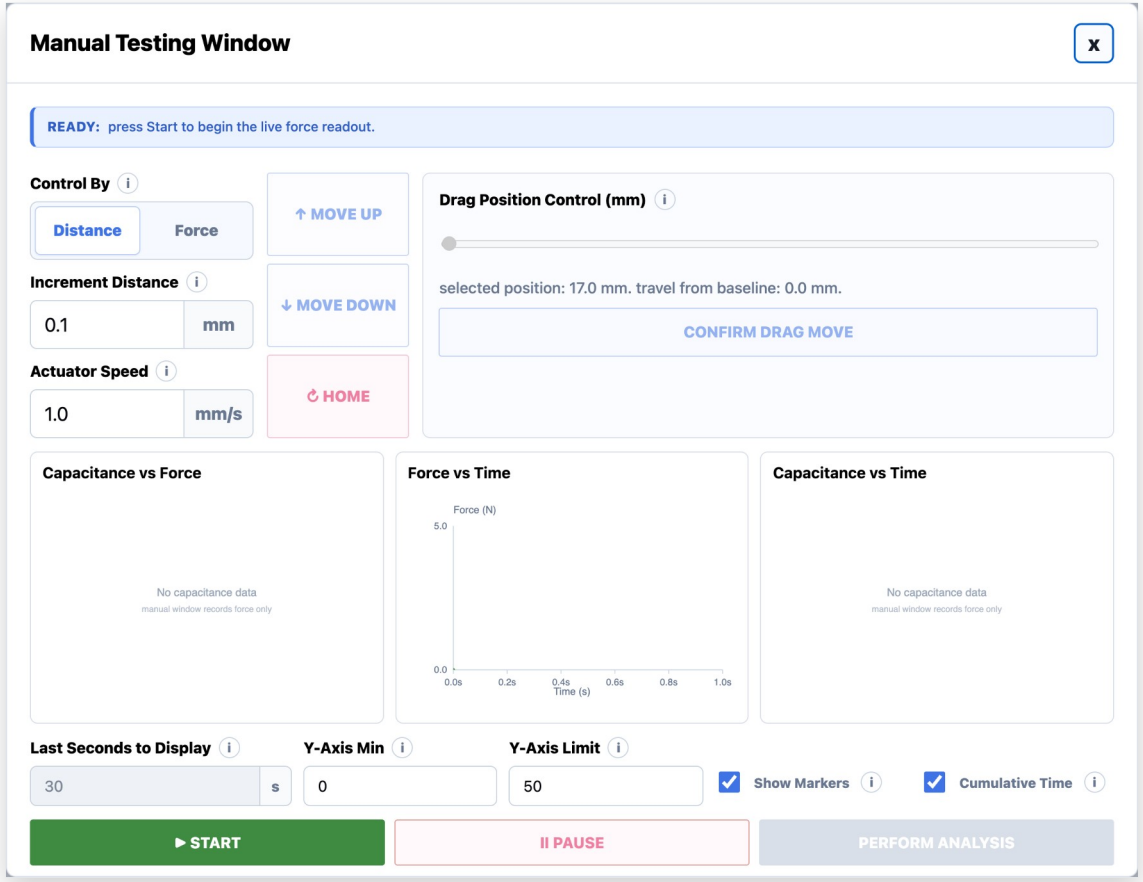
  Power users type a safe expression; live preview and validation, auto-scaled into the bounds.



Mockup, not yet wired into the app. The actuator speed limit is a placeholder until measured on the rig.

DEMO

Manual testing window



- **Distance or force control**

  Jog by a set distance, or drive until the load cell reads a target force.

- **Compress and decompress**

  Closed-loop in both directions - press to a force, then release back to one.

- **Continuous force recording**

  Force vs time records from the moment the window opens.

- **Editable while moving**

  Graph controls stay live mid-move; motion controls lock for safety.

DEMO

EM test & run management

- **Live state machine**

  Every run moves through a strict set of states, each one timestamped in the status log.

- **Automated multi-run**

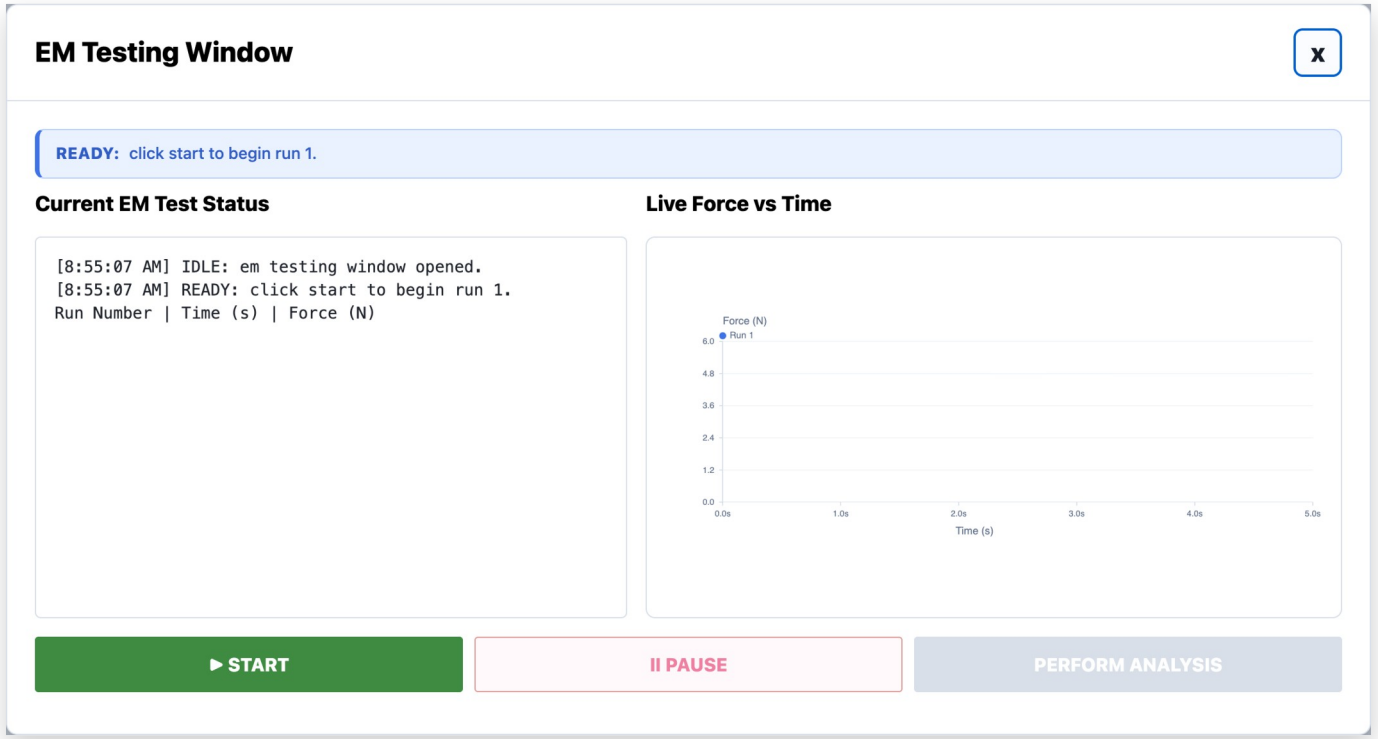
  Runs the configured presses back to back, auto-pausing so the sensor can be reset.

- **Redo without data loss**

  A redo creates a new superseding run - the original is never overwritten or deleted.

- **Homes first, every time**

  Drives to the true baseline before pressing, never starting from an unknown spot.



DETAIL

Calibration & Fuji film

1

Operator-set extrusion distance

A new field controls how far the actuator extrudes for the Fuji-film press, capped at 12 mm and remembering the last value used.

2

Fine manual jog

Step the actuator in small increments to fine-tune position before a calibration press.

3

Press to a calibrated target

The Fuji press drives down until the load cell reaches its target force, then stops.

4

Reliable jog state

Fixed an intermittent bug where the window thought the stage was still moving after a jog finished.

SAFETY

Built into every test

Force ceiling

A hard over-force limit stops the actuator before the load cell can be overloaded.

Spike detection

A sudden force jump - metal-on-metal contact - halts motion immediately.

Travel limits

The stage never drives into its mechanical end stop.

Disconnect handling

Losing the actuator or load cell mid-test stops, retracts to home, and invalidates the run.

Snap-to input limits

Out-of-range entries snap to the nearest limit and say which limit they hit.

Single-owner load cell

Exactly one reader touches the device, so it can't be corrupted mid-session.

RESILIENCE

Surviving a disconnect

Stop and retract home

Any actuator or load-cell drop halts motion and backs the rod off the sensor, never leaving it pressed.

Load cell self-recovers

The reader re-initializes the device in place, so a reconnected cell reads again with no app restart.

Never fakes the force

On a real rig a missing load cell blocks the run instead of quietly recording simulated data.

Fast, honest detection

A stalled reader is caught in a fraction of a second, so the run aborts and the actuator stops promptly.

Right message per sensor

A load-cell drop and a Zaber drop show different reconnect guidance, not one generic error.

Live COM port list

A cable plugged in after the app opens shows up in the port picker on its own.

The load-cell recovery behavior is verified in simulation and by code review; the real-device reopen still needs a bench test on the Windows rig.

REFERENCE

Limits, bounds, and safety stops

Operator inputs - snap into range on commit

Manual position	**17 to 50.8 mm**
Target force (manual)	**0 to 32 N**
Increment distance	**0.1 to 12 mm**
Actuator speed	**0.01 to 2 mm/s**
Fatigue force bounds	**up to 32 N, upper > lower**
Fatigue frequency	**0.1-5 Hz (square 0.1-1)**
EM runs per test	**1 to 10**

Fixed physical and safety limits

Travel range	**17 to 50.8 mm**
Force ceiling (safety)	**33 N: stops + homes**
Actuator peak thrust	**25 N hardware max**
Sudden-jump cutoff	**speed-scaled, 1.5 to 500 N**
Lost connection	**safe stop, auto re-home**
Force sampling	**100 Hz**

Snap-to rounds each field to its precision and clamps it into range on commit, so you can still type freely while editing. EM run target (32 N) and Fuji target (20 N) are where a good press stops, not limits.

PART 2

Under the hood

ARCHITECTURE

How it's built

Browser GUI

Vanilla HTML / CSS / JS - one file per test tab (em.js, manual.js, fatigue.js, shear.js, calibration.js), plus shared.js and main.js

↓ JSON over local HTTP · the UI polls for live status

Local Python server + test engine

ThreadingHTTPServer (server.py) and run_engine.py - one background thread per test, polled for live status, a single-owner load-cell reader, and a simulation fallback

↓ serial commands · .NET driver calls

Hardware drivers

Zaber actuator over serial (zaber-motion) and the FUTEK load cell over .NET (pythonnet, Windows only)

Packaged as a pywebview desktop window, built into a single .app / .exe with PyInstaller - no terminal, no setup.

ENGINEERING

The hardest bug: the load cell

1

Problem

The load cell kept dropping to simulated data partway through a session - and once it dropped, every test after it read fake values.

2

Diagnosis

The manual window's live graph and the active test were reading the device at the same time, on different threads. The driver can't be read twice at once, so it corrupted.

3

Fix

Re-architected to a single-owner reader: one background loop owns the device and everything else reads its value. Two simultaneous reads are now impossible, in every window.

TIMELINE

Day by day

Days 1-18 went to clinical data validation (OHSU / HFH cases, the MATLAB pipeline, and site decks). The fixture GUI begins on Day 19.

Day 19 · Jun 10	Annotated the test engine (run_engine) end to end - one code path for simulation and real hardware
Day 20 · Jun 11	Reliable motion + serial - killed the 12 s move lag, stopped false disconnects, fixed the 17 mm reference
Day 21 · Jun 12	Safety UI state - the waiting pill, control locking, safety-stop homes; redid the calibration window
Day 22 · Jun 15	Run management + reporting - made the report numbers match the graphs; the redo-run workflow
Day 23 · Jun 16	Other track - UCI data-parsing script and website planning (no GUI)
Day 24 · Jun 17	Real-hardware behavior - true home each test, Shear wired to the load cell, manual force-feedback
Day 25 · Jun 18	Load-cell reliability - the single-owner reader, plus fatigue and EM safety fixes
Day 26 · Jun 19	Other track - competitor research and a clinical-affairs interview (no GUI)
Day 27 · Jun 22	Snap-to input limits, the limits and bounds reference, and user-story reconciliation
Day 28 · Jun 23	Formal test plan done - 45 EM test cases across functional, negative, interruption, boundary, and journey paths
Day 29-30 · Jun 24-25	Connection gating and deferred EM run files, square-wave characterization, shear CAP-file prompt, travel limit to 41 mm
Day 33-36 · Jun 30-Jul 3	Interactive live graphs and Manual Start button; disconnect recovery, retract-to-home, and load-cell reopen; waveform-dependent fatigue frequency; live COM port list

DOCUMENTATION

It's all tracked in one workbook

Every requirement, change, and check lives in a single documentation workbook, so nothing is only in someone's head.

Section	What it documents	Size
Verification checklist	Per-area pass/fail checks: when it happens, pass condition, status, verified	84 checks
GUI Updates Log	Every visual, backend, hardware, and documentation change, with files touched and test status	60+ entries
User Story Tracker	Feature requirements with priority, status, and acceptance criteria	100+ stories
Limits and bounds	Every input range and fixed physical / safety limit, per test	reference
Testing info + folder workflow	Test setup notes and the analysis save workflow	reference

Across 14 sheets. The checklist drives the formal test pass; the updates log is the change history.

DOCUMENTATION

The documentation folder

Beyond the tracking workbook, the reader-facing docs are organized into six guides - each written for a specific reader.

Document	What's inside
1 · Project overview	What the GUI is, the goals it serves, and the scope of each test workflow
2 · Operator guide	Step by step: how to set up, run, and analyze each test at the bench
3 · Technical reference	Every limit, bound, safety stop, and force formula, per test
4 · Developer guide	Architecture, code layout, and how to build and extend the app
5 · Test plan and verification	Formal test cases plus the pass/fail verification checklist
6 · Design concepts and proposals	Mockups, future-feature proposals, and the design decisions behind them

Formal test cases done for 7 of 11 features - 4 to go.

STATUS

Where it stands & what's next

Solid now

- Every test type runs end to end
- Safety behaviors verified in simulation
- Verified on the rig where hardware was available
- Documentation organized into 6 guides
- Packaged and runnable without a terminal

Next

- Finish the formal test cases - 7 of 11 features done, 4 to go
- Confirm safety-critical cases on the real rig
- Reconcile the user-stories doc to the current GUI

RESOURCES

Documentation & code

User stories

11 features, 100+ stories - every happy and unhappy path, safety handling, and a verification method for each.

[add link]

Test plan

Formal test cases traced to each story, organized by technique and tagged simulation vs real rig.

[add link]

Source code

The full GUI, test engine, and hardware drivers, version-controlled.

github.com/jacqchuang09/test_fixture

Thank you · Jacqueline Chuang

APPENDIX

Deep dives & backup

ANTICIPATED QUESTION

Is the live graph behind the real force?

During a fast fatigue cycle (up to 5 Hz), the on-screen trace can sit a few tens of milliseconds behind the true load. That is real, so here is why it does not put the test at risk.

Safety never looks at the graph

Spike and force-limit cutoffs run on the raw sample stream in the backend, before any point is drawn. Display lag cannot delay a safety stop.

The actuator follows the backend, not the screen

The waveform is generated and driven in Python. The live graph is a read-only mirror of that same stream.

So the lag is only a UX number

It changes how live the graph feels, never whether the test is safe or the saved data is correct.

TOOL

Live graph latency simulator

A standalone page (opens in any browser, no backend) that models how far the live graph trails the real force, so the number is honest instead of a guess.

- **Models the whole path**

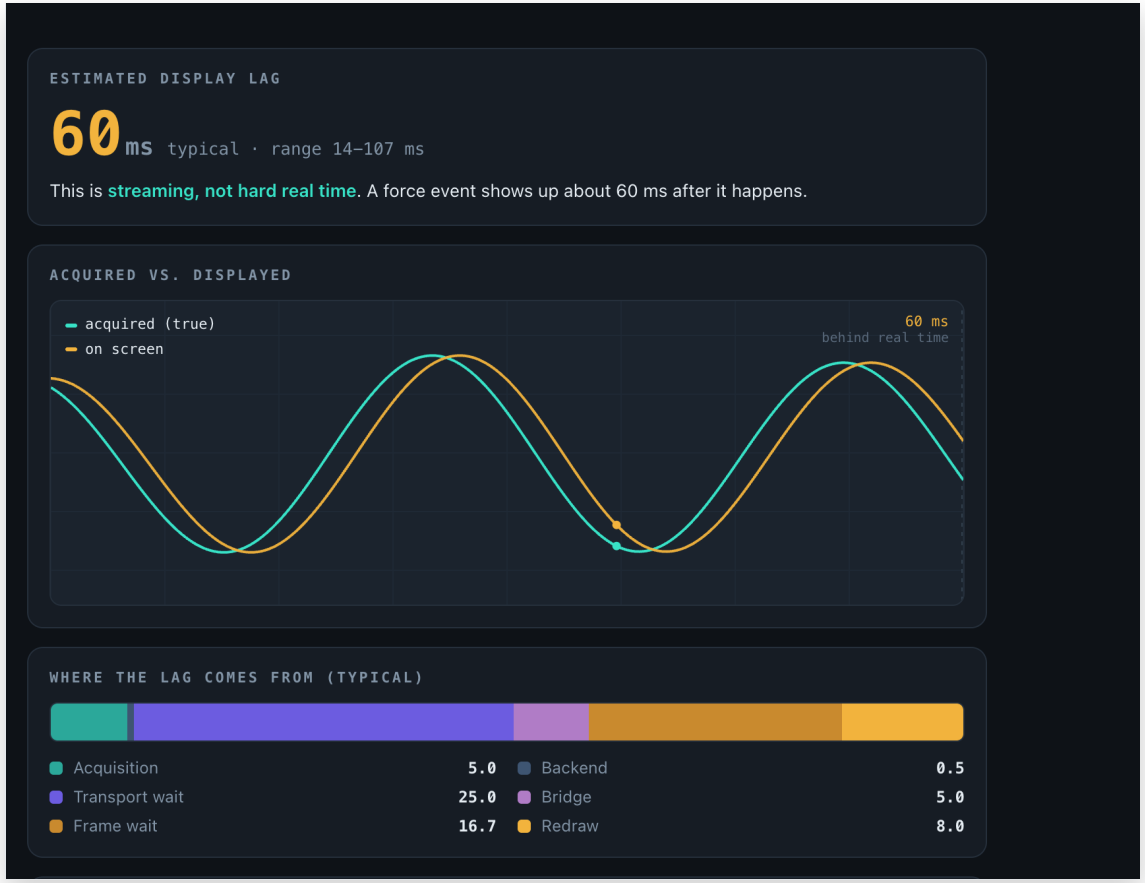
  Load cell sample, backend read, pywebview bridge, then redraw. Each stage is a slider you can dial in.

- **Gives a ballpark**

  A typical setup lands near 60 ms (range 14-107 ms): streaming, not hard real time.

- **Measure the real number**

  Stamp perf_counter() in Python and performance.now() at draw, then report the median and 95th-percentile gap over a 30-60 s run.



DOCUMENTATION

Live force graph: values

Current values for the live force display. Estimates for now; replace with measured numbers once profiled on the rig.

Property	Value	Notes
Y axis	Force (N)	live load-cell reading
X axis	Time (s)	scrolling window
Sample rate	100 Hz	one reading every 10 ms
Display lag (typical)	~60 ms	acquisition to drawn (estimate)
Display lag (range)	14 to 107 ms	best to worst case (estimate)
Update cadence	~10 / second	browser polls the status endpoint
Data source	raw backend stream	safety reads this, not the graph

DOCUMENTATION

Live force graph: lag breakdown

Stage	Typical (ms)	What it is
Acquisition	5.0	half a sample period
Backend read	0.5	per-sample processing
Transport wait	25.0	half the push interval
Bridge	5.0	Python to JS hand-off
Frame wait	16.7	half a redraw frame
Redraw	8.0	paint one frame
Total (typical)	~60	sum of the stages

How the typical figure is built:

```
typical = sample/2 + backend + batch/2 + bridge + frame/2 + redraw
        = 5.0 + 0.5 + 25.0 + 5.0 + 16.7 + 8.0   ~=   60 ms
```

DOCUMENTATION

Fatigue simulator: force calculation

How a shape becomes the force the actuator targets, cycle after cycle.

Step	Calculation	Result
Shape	f(phase), phase 0 to 1	the waveform you picked or typed
Normalize	(v - min) / (max - min)	shape squashed to 0 to 1
Scale to bounds	lower + (upper - lower) * shape	force in newtons
Top-off (clamp)	min(force, 33 N ceiling)	never exceeds the safety ceiling
Repeat	one cycle per 1 / frequency	thousands of cycles

In code:

```
shape = normalize(f(phase))                  // 0 to 1
force = lower + (upper - lower) * shape       // newtons
force = min(force, FORCE_CEILING_N)           // 33 N top-off
```

DOCUMENTATION

Fatigue simulator: round-off check

Whether the actuator can physically follow the curve, or whether the sharp parts round off on the rig.

```
period  = 1 / frequency             // seconds per cycle
dt      = period / samples          // time between points
slope   = abs(f[i] - f[i-1]) / dt   // newtons per second
maxSlew = max(slope over one cycle) // steepest demand
limit   = stiffness * max_speed     // actuator ceiling (placeholder 60 N/s)
verdict = maxSlew <= limit          // green, else amber: rounds off
```

Worked examples (same 2 to 20 N bounds):

Waveform	Needs	Verdict
Sine, 1 Hz	~57 N/s	green, follows faithfully
Blood pressure, 60 bpm	~230 N/s	amber, upstroke rounds off
Square	very large (instant jump)	amber, comes out like a sine

limit = block stiffness (N/mm) x max actuator speed (mm/s). The 60 N/s is a placeholder until the block stiffness is measured on the rig.

DOCUMENTATION

Fatigue square-wave frequency limit

A rig frequency sweep (tools/zaber_square_wave_characterization.py) measures where a square wave stops holding its flats. Flat-hold - the fraction of each half-cycle held at the commanded level - is the decider: a true square holds most of it, a triangle almost none.

Frequency	Flat-hold	Reaches bounds	Shape
≤ 0.25 Hz	50-81%	yes (100%)	square - holds a flat
0.3-0.45 Hz	13-42%	yes (100%)	rounding into a triangle
0.5 Hz	~1%	barely (98%)	triangle - no flat
≥ 0.75 Hz	0%	no (60-46%)	cannot reach amplitude

Faithful square: keep it at or below 0.25 Hz. The frequency field allows a square up to 1 Hz, but above ~0.25 Hz the wave rounds into a triangle and above ~0.5 Hz it no longer reaches its bounds. The rod slews a 2 mm swing in about 1 s and lags the command ~165 ms - so the measured limit lands far below the naive speed-only ceiling of a few hertz (which wrongly assumes the rod reaches full speed instantly).